Silent Failures in On-Device Speech Recognition: A Measurement Study of CPU Dictation and Long-Form Meeting Transcription

Mohsen Seyedkazemi Ardebili
Responsible Systems Lab
mohsen.seyedkazemi@gmail.com

October 2026

Abstract

On-device speech recognition has no provider to monitor its output or offer a second opinion. We measure a shipping open-source dictation and meeting-transcription system and report failures that return successfully and are poorly summarised by one corpus word error rate (WER). On 200 LibriSpeech *test-other* utterances, 10 default-configuration decodes of Whisper `large-v3` on one machine type span 4.84–7.69% WER, so one decode cannot rank it against smaller checkpoints. In every archived `large-v3` run the aggregate substitution and deletion counts are identical and only insertions move. Disabling the temperature fallback made the output deterministic but raised WER to 15.26%, so it is not a fix. Disabling conditioning on previous text appears to lower WER by 1.7 points, but paired per utterance the gain is 1.05 points with an interval spanning zero; three clips account for 95.7% of the 47 errors removed (gross, before 8 errors added elsewhere). Beam-width conclusions did not transfer across the cells tested, and LibriSpeech cross-family comparisons are confounded by training-data overlap. On 16 AMI meetings diarization error is 26.71% (27.37% time-weighted; threshold tuned on the same corpus); fixing the speaker count gave no resolvable difference. A 41-minute meeting was reported as successfully transcribed although its transcript was dominated by one repeated phrase. A preliminary live-versus-batch word-count check flagged it (16.0×, against 1.008× on one healthy meeting); the check was fitted and assessed on the same five meetings, with counts for two. Harness and results are public; manuscript scripts are not.

1 Introduction

On-device speech recognition has become practical. A commodity laptop CPU now decodes speech several times faster than real time, model weights fit in a few hundred megabytes, and a growing class of users — clinicians, lawyers, journalists, people working offline or under data-residency rules — want dictation that never sends audio anywhere. We built and shipped such a system, YazSes [24, 25], and described its architecture, privacy model and first evaluation in a companion paper.

That paper measured what a systems paper usually measures: word error rate on clean read speech, real-time factor, per-stage overhead. This one reports what we learned when we kept measuring — across two corpus splits, five Whisper checkpoints, three inference engines, four CI runner environments spanning two ISA families, sixteen AMI meetings (four series, 12 of them scenario-based), and 141 archived repeat-probe decodes (run records in 14 artifacts, each of 40 to 200 utterances) — and it reaches a different conclusion than we expected.

The thesis. *Some of the most damaging failures in on-device speech recognition are silent, and a single corpus-level accuracy figure is a poor instrument for seeing them.* A cloud provider observes millions of requests, retains audio, and can compare a suspicious transcript against a second model. A privacy-preserving local system has, by design, no fleet-level telemetry and no provider-side second opinion, and a user who cannot tell a confidently wrong transcript from a right one. Short-form dictation produces a single local decode; the meeting path produces two, whose disagreement is an additional but limited consistency signal. Every decoding and pipeline failure reported in §§4–11 returned successfully (the dependency failures of §12 surface as install errors). Several return a different answer each time they are asked. One deleted the only recording of a 41-minute meeting because it believed it had transcribed it; a rolling live transcript survived, but nothing surfaced it.

Contributions. Each, except the long-form incident of §11, is backed by a committed artifact whose provenance block names the machine and the library versions. The corpus subset is identified by the artifact's selection rule and size; only some later probe artifacts also store a digest of the selected ids, and the CI-runner artifacts store neither (§7).

1. **One decode cannot rank the largest checkpoint, and across families the standard benchmark confounds the comparison** (§4). Over 8 engine–checkpoint configurations with bootstrap confidence intervals, `large-v3` returns 4.84–7.69% across 10 default-configuration decodes — below both smaller English-only checkpoints in 4 of them and above them in the rest — at 4.8× `small.en`'s decode time. A Token-and-Duration Transducer [31] scores 1.9× lower than `medium.en`, but its publisher lists LibriSpeech among its training data while Whisper is evaluated on it zero-shot; we treat that comparison as evidence about the *benchmark*, not about the engines.
2. **The same audio decodes to different text, and the variation is in inserted words** (§5). Five repeats of `large-v3` give 5 distinct transcripts; on that checkpoint the aggregate substitution (87) and deletion (15) counts never move, in any arm or run. We localise the variation to the interaction of two decode parameters, show that the sign of the conditioning effect differs across the checkpoints tested, and show that the parameter acts on ordinary short utterances.
3. **A corpus mean can be a false description of what a user experiences** (§6). The configuration that looks 1.7 WER points better is better on 4 utterances and worse on 8; its gross error reduction is 95.7% carried by three clips (45 of 47 errors removed; 8 errors are added elsewhere, leaving a net of 39), and its paired size (1.05 points) is smaller than the 1.7 points obtained against the five-repeat mean. We report the tail argument and retract the mean one.
4. **Accuracy spread across CI runners shrank as checkpoints grew, in the three checkpoints compared; timing does not transfer** (§7). Identical code on 4 runner environments (two ISA families) spreads 0.49 WER points on `tiny.en` and 0.0 on `small.en`. The largest `tiny.en` spread is between the two x86_64 runners, so these data do not isolate the instruction set.
5. **Decode-parameter conclusions did not transfer between the cells tested** (§8). Beam width matters on the dictation checkpoint ($p = 0.0026$ on *test-other*, 0.013 after Bonferroni correction over 5 comparisons) and is unresolved on the checkpoint the latency governor switches to ($p = 0.4118$) — so a policy tuned on one measures a combination the product never runs.
6. **Two negative results, reported** (§9). Of 32 silence-lead-in comparisons, 4 are nominally significant at $\alpha = 0.05$ and **none** survives a Bonferroni correction [8]; and streaming decode

leaves 0.0% of text visible at key-release on the shipping checkpoint, arriving *later* than the batch path it would replace.

7. **Diarization on real meetings, and a configuration trap** (§10). DER 26.71% unweighted / 27.37% time-weighted over 16 AMI meetings (4 series: fewer independent groups than recordings); pinning the speaker count makes the tuned clustering threshold inert (verified by bit-identical output), and the resulting mean difference is not resolvable (sign test $p = 1.0$), nor is it uniform across series.

8. **A silent long-form decode collapse, and a cross-decode consistency check** (§11). A finished meeting reported success with a 284-word transcript dominated by one phrase repeated 93 times. The strongest signal is the ratio between the word counts of two differently configured decodes of the same audio, 16.0× on the collapsed meeting and 1.008× on one healthy meeting; the operational alarm uses a preliminary $\geq 3\times$ threshold.

9. **Dependency resolution blocked some install targets that no accuracy table shows** (§12). Of 92 (feature, platform) install targets (22 optional feature sets plus the base install, on 4 platform targets), 8 cannot resolve — 6 against the operating-system floor the resolver assumed and 2 because a dependency's last release for that architecture predates our own floor. The check is static: it says nothing about runtime compute limits, which we did not assess.

What this paper is not. It is not a new model, a new decoding algorithm, or a user study. It measures a shipping artifact (30,306 lines of Python, 1,639 test functions, 147 decision records, as counted at version 2.12.0.dev4 on 2026-07-30; the measurements themselves ran on versions 2.29.0–2.30.0, and the system has grown since) with the intent of telling other builders of on-device speech systems which of their assumptions we found to be false. In the taxonomy of Wobbrock and Kientz [30] the companion paper made an *artifact* contribution; this one makes an *empirical* contribution about that artifact's failure modes, and still not one about its users (§15).

2 Background and Related Work

On-device ASR. Whisper [23] made general-purpose speech recognition available as a downloadable checkpoint, and inference engines such as CTranslate2 [20] (via faster-whisper [27]) made int8 CPU decoding fast enough for interactive use. Two other families are now competitive on CPU and are measured here: Moonshine [12], designed for live transcription and voice commands on constrained hardware, and Token-and-Duration Transducers [31], whose duration prediction lets the decoder skip input frames. Published accuracy figures come from the authors' own reference setups; we measure CPU configurations a user can run, rather than assuming they inherit the published numbers.

Hallucination and repetition in ASR. Koenecke et al. [15] found that 1.4% of transcriptions of aphasic and control speech by the hosted OpenAI Whisper API contained hallucinated phrases ("roughly 1%" in their abstract), and that the harms are not uniformly distributed across speakers. Whisper's own paper describes the temperature fallback and previous-text conditioning that this paper manipulates, and notes that beam search reduces the repetition looping that is more frequent under greedy decoding [23]. Later work attributes hallucination to non-speech audio [2] and to drift and repetition on long-form audio, mitigated by voice-activity-based segmentation [1]. Our §5 is a complement at the level of a repeated, identically configured decode: in the aggregate error counts of the runs measured here, all of the run-to-run variation is in insertions, and the two

decode parameters whose interaction produces it can be isolated. The failure mode resembles the degeneration described for open-ended neural text generation [11] — a decoder conditioned on its own output falling into a repetition loop — arriving in a setting where the output is supposed to be grounded in audio.

Confidence and error detection. Detecting that a recognizer is wrong without a reference is an old problem with its own literature: Jiang's survey [13] covers confidence measures computed from a single decoding pass, from acoustic and language-model scores to word posteriors. Nothing in this paper claims that error detection *requires* a second decode. §11 studies one cheap extra signal — agreement between two decodes the system already performs — and does not compare it with confidence-based detectors, which we did not implement or measure.

Streaming. LocalAgreement, a partial-hypothesis policy introduced by Liu et al. [16] and applied to Whisper by Macháček et al. [17], emits a prefix once successive decodes agree on it. It is designed for unsegmented long-form speech; §9 measures it in the different regime of a short push-to-talk burst on CPU, where we find it does not pay.

Diarization and its scoring. The diarizer measured here composes pyannote's powerset segmentation model [22] with ERes2Net speaker embeddings [6] through sherpa-onnx [14]. Diarization error rate and its scoring conventions come from the NIST Rich Transcription evaluations [9]; we evaluate on the AMI Meeting Corpus [5] and on VoxConverse [7]. We report both an unweighted per-recording mean and the time-weighted figure that `md-eval` computes, because the second is the one comparable with published tables under the same protocol, and §10 shows they differ by more than a rounding error on a corpus whose recordings differ in length.

Measurement studies of deployed systems. Methodologically this paper belongs less with ASR modelling work than with the measurement literature that takes a running system as its object and asks what its published numbers fail to capture. We inherit two conventions from it and state them because they are unusual in speech papers: nearly every result is a committed artifact with a machine-readable provenance block (the exception is §11), and a result that contradicts an earlier claim of ours is published beside it rather than replacing it silently (§6, §10, §4). We did not find a comparable failure-mode study of an on-device speech system in a non-exhaustive search; the closest work in spirit is the hallucination-harms analysis above [15], which studies outputs where we study decode configurations and the pipeline around them.

Statistics. WER differences are compared with a paired bootstrap over utterances following Bisani and Ney [4]; families of comparisons are Bonferroni-corrected [8] where stated. Scoring uses the Whisper English text normalizer [3] and JiWER [29] throughout, applied identically to reference and hypothesis.

3 Method

The system under measurement. YazSes is an open-source (Apache-2.0) hold-to-talk dictation daemon for Linux, macOS and Windows that also provides two long-form paths: an offline file-to-text importer with optional diarization, and a hands-free meeting recorder. All decoding happens on the local CPU. The architecture is described in the companion paper [24]; this paper treats the system as the object of measurement rather than the contribution.

Corpora. Short-form accuracy uses LibriSpeech [21] *test-clean* and *test-other*, 200 utterances per split, drawn by a deterministic speaker-stratified round-robin across all speakers of the split (33 speakers and 23.4 minutes of audio for *test-other*). The selection is reproducible without an RNG. The artifacts record the selection rule and the number of utterances; only some later probe artifacts also store a SHA-256 digest of the selected utterance ids *in decode order* (because one utterance's decode can depend on what preceded it), and the engine-matrix, beam, silence-lead-in, streaming, diarization and CI-runner artifacts do not. The diarization artifacts name each recording by its corpus id. Long-form work uses 16 meetings of the AMI test split [5] (headset mix, human-annotated references, 8.5 scored hours) and 15 recordings of VoxConverse [7]. The 16 AMI recordings are not independent: they are the four meetings (a–d) of each of 4 series (EN2002, ES2004, IS1009, TS3003), and under the corpus's meeting-id scheme [28] three series are scenario-based, in which participants play assigned roles in a design task, and one (EN2002) is naturally occurring. That is 12 scenario recordings and 4 natural ones, from 4 groups; §10 reports the series structure.

Metrics. WER is computed by JiWER [29] after applying the Whisper English normalizer [3] to both sides. Real-time factor (RTF) is decode time over audio duration; RTF < 1 is faster than real time. DER is frame-based with a Hungarian one-to-one speaker mapping, following `md-eval` semantics [9]; we report it both strict and with a 250 ms collar, and both as an unweighted mean over recordings and time-weighted over total scored speech. The repository's scorer was checked against NIST `md-eval-22.pl` and `pyannote.metrics` on the same hypothesis RTTMs.

Statistical treatment. Every accuracy comparison for which we make an inferential claim is *paired* on the utterance (or the recording) and, where the archived artifact provides one, reported with an interval (§6, §10, and the headline comparison of §8), not as a difference of two means. Exceptions: the secondary beam-width p-values of §8 are given without their intervals (the intervals are in the public artifacts), the silence-lead-in grid (§9) reports exact McNemar p-values without an interval, and descriptive comparisons of point estimates (for example between rows of Table 2, or the checkpoint ladder of §5) are labelled as such. WER differences use a paired bootstrap with 10,000 resamples [4]. Where a comparison is one of a family, the family is stated. A Bonferroni correction [8] is applied to the silence-lead-in grid (§9); for the beam-width grids (§8) we report the raw bootstrap p and the Bonferroni-adjusted value, because the raw values alone would overstate what the grid resolves. Intervals and p-values are descriptive of these samples and are not adjusted across sections. Where a difference is carried by a handful of items, we say so and give the concentration, because a corpus WER is a weighted mean and one runaway utterance moves it a long way (§6).

Provenance and archiving discipline. Every measurement in this paper except the long-form incident of §11 (a private recording, documented only in the project's release notes, source and tests) is a committed JSON artifact carrying the CPU model, logical CPU count, OS, kernel, Python, library versions and one-minute load average, plus an invoking command line (with home directories and login names redacted) and a corpus digest *where recorded*: older artifacts (for example the engine matrix) lack one or both, and 48 of 80 lack the command line (Availability). The versions recorded are those of the packages the harness reports, not of every dependency: the diarization artifacts record `faster-whisper` and `ctranslate2` but *not* the installed version of sherpa-onnx, which is the diarizer under test, nor the revisions of its two model files. The release lockfile of the recorded `yazses` version (2.29.0) pins sherpa-onnx 1.13.4, which is probably what ran but is not a record of it. Two numbers are comparable only if the machine matches. Three rules were adopted after they were violated: writing a result never overwrites an old one (the displaced file moves to a

Table 1: Where every measurement in this paper was taken, with the one-minute load average recorded at the time. Generated from the provenance blocks.

Section	Measurement	Host	Load (1 min)
§4	engine matrix	Xeon 8573C (rented, 16 vCPU)	16.0
§5	decode arms, mechanism	Xeon 8573C (rented, 16 vCPU)	5.2
§6	per-utterance re-scoring	Xeon 8573C (rented, 16 vCPU)	3.8
§7	cross-ISA matrix (x86 leg)	CI runner (4 vCPU)	0.4
§7	within-host repeat	i7-1370P (laptop, 20 vCPU)	26.4
§8	beam grids	Xeon 8573C (rented, 16 vCPU)	8.4
§9a	silence lead-in	Xeon 8573C (rented, 16 vCPU)	9.7
§9b	streaming	i7-1370P (laptop, 20 vCPU)	24.1
§10	diarization DER	Xeon 8573C (rented, 16 vCPU)	1.0
§10	attribution guard	i7-1370P (laptop, 20 vCPU)	3.5
§12	resolution matrix	i7-1370P (laptop, 20 vCPU)	13.2

history directory), a file whose name ends `-significance` is an *analysis* of a measurement rather than a second measurement, and a partially failed matrix records its failures rather than scoring the subset that survived. The exploratory runs are archived too, including the ones that went nowhere, because the ranges that were wrong are the part a later reader cannot reconstruct.

Hosts. Table 1 names the machine and the one-minute load average behind every section, generated from the artifacts' own provenance blocks so it cannot drift from the runs. Timings are properties of the machine and are never merged across hosts. Two rows deserve to be read before the sections that depend on them: the engine matrix in §4 was taken on a host at essentially full utilisation, so its RTF column describes a common host and harness during one contended matrix run, with per-row background contention not controlled. The values are exploratory, not an idle-host or cross-platform latency ranking; and the streaming comparison in §9 — a pure latency claim — was taken on a contended laptop, which §9 restates where it matters.

Reproduction. The benchmark harness and the JSON artifacts are public (see Availability). The manuscript-specific script that turns the artifacts into the macros and tables of this paper is a private working file, so a reader can re-derive each value from the public artifacts but cannot re-run our generator. The only values not read from an artifact are the long-form incident figures of §11, transcribed from the shipped source's own record of it.

4 One Decode Cannot Rank the Largest Checkpoint, and the Benchmark Confounds Rankings Across Families

Table 2 reports 8 engine–checkpoint configurations on the same 200 *test-other* utterances, same normalizer, same scorer, one run on one host (all rows share it), CPU only, no GPU.

Three things in that table undercut the default assumption that a bigger checkpoint is a better one — but none of them in the way the table's own numbers first suggest, and two of them undercut the table itself.

The largest Whisper checkpoint does not have one accuracy, it has a distribution — and that is the finding. The `large-v3` row reads 7.69%, worse than `small.en` (5.59%) and

Table 2: Accuracy and speed of 8 on-device configurations on 200 speaker-stratified LibriSpeech *test-other* utterances (33 speakers, 23.4 min), CPU, INTEL(R) XEON(R) PLATINUM 8573C at the load recorded in Table 1. Precision: the `faster-whisper` rows record `int8`; Parakeet was loaded through onnx-asr with int8 requested (the precision actually loaded is not recorded); Moonshine runs the ONNX weights as distributed (package `useful-moonshine-onnx` 20251121; the weight revision is not recorded). Intervals are percentile bootstrap 95% CIs (1,000 resamples) over utterances for each row; RTF is decode time over audio duration. **The second column is load-bearing:** it records whether each publisher states that the LibriSpeech training split was used [23, 12]. Whisper's authors evaluate zero-shot, which excludes that split but is not an audit of their web-scraped corpus. Rows differing in the column are not a like-for-like comparison; compare within a value of it, not across it. † `large-v3` is the one row whose configuration does not reproduce: this is a single draw, and 10 default-configuration draws of it span 4.84–7.69%. It cannot be ranked against the rows above it from this value — see §4, which is about why this table has to be read that way.

Engine / checkpoint	LS train split used?	WER (%)	95% CI	RTF
faster-whisper `tiny.en`	no (zero-shot)	11.77	[9.86, 14.02]	0.034
faster-whisper `base.en`	no (zero-shot)	9.46	[7.92, 11.17]	0.049
faster-whisper `small.en`	no (zero-shot)	5.59	[4.44, 6.8]	0.104
faster-whisper `medium.en`	no (zero-shot)	5.51	[4.39, 6.72]	0.272
faster-whisper `large-v3`†	no (zero-shot)	7.69†	[3.52, 14.63]	0.49
Moonshine `tiny`	**yes**	10.35	[8.74, 12.0]	0.017
Moonshine `base`	**yes**	8.04	[6.68, 9.47]	0.023
Parakeet TDT 0.6B v2	**yes**	2.88	[2.2, 3.63]	0.053

`medium.en` (5.51%). **Do not rank it from that number.** It is one draw from a configuration §5 shows is not reproducible. The archive holds 10 decodes of the identical default configuration on this split — this table's, 4 from an earlier instability probe and 5 from the determinism probe: 4.84, 5.11, 5.35, 5.46, 6.07, 6.1, 6.21, 6.53, 6.61, 7.69%. They share a CPU model, operating system and library versions (faster-whisper 1.2.1, CTranslate2 4.8.1), but not everything: they were taken on different occasions at one-minute load averages of 15.97 (this table), 3.95 and 5.21, under YazSes versions 2.29.0 and 2.30.0, and the provenance blocks do not identify the instance, so we claim "same machine type" and no more. (An earlier engine-matrix run of this configuration scored 4.86%, but its artifact was overwritten before archiving and is not counted.) The draws span 2.85 points, and in 4 of 10 the WER is below both smaller English-only checkpoints' table values rather than above them.

So the claim this table cannot support is "the biggest checkpoint is less accurate". The claim it does support is sharper and worse for the user: **on CPU the biggest checkpoint costs 4.8× the decode time of `small.en` (under the load of Table 1) and, in our decodes, returned accuracies ranging from better than either smaller model to clearly worse**, with no way to know in advance which decode you will receive. A single figure for a configuration whose repeated decodes span 2.85 points is not a rank, and printing one — as this table does, and as single-decode benchmark rows generally do — is the error the rest of this paper is about. We leave the row in place, marked, because removing it would hide the point.

The confidence interval says the same thing from the other side. $[3.52, 14.63]$ is 11.1 points wide: 2.7× the next widest row and 7.8× the narrowest. A bootstrap resamples utterances within *one* decode, so that width reports a heavy per-utterance tail, which §6 examines directly — and §5 then adds the second, independent variance across decodes that no interval in this table captures. The

smaller English-only checkpoints show neither problem under the shipping configuration: `small.en`, `medium.en` and `base.en` each return one hash across 5 repeats and reproduce their table values exactly, so their point estimates are stable. Stability is not resolution, however: `small.en` and `medium.en` are 0.08 points apart with heavily overlapping intervals, and we did not run a paired test between them. One further asymmetry is worth naming: `large-v3` is multilingual where the others are English-only, and an English-only checkpoint has an easier task on English.

The cross-family row is a benchmark confound as much as a result, and we report it as one. Parakeet TDT 0.6B [31, 19] run through onnx-asr [26] (version 0.12.0) scores 2.88% [2.2, 3.63] at 18.9× real time — a word error rate 1.9× lower than `medium.en` at 5.1× the speed. It is the most accurate row and decodes faster than every Whisper checkpoint from `small.en` up, though the Moonshine models and the two smallest Whisper checkpoints decode faster still.

We do *not* conclude from this that the transducer is the more accurate model. Three different things are easy to conflate here. (i) *Training on the dataset family:* the model card for Parakeet lists the 960-hour LibriSpeech training set among its human-transcribed data [19] — a statement about the exact checkpoint measured, which the architecture paper [31] does not make — and initialises the encoder from a self-supervised model pretrained on LibriLight (LibriVox audiobooks), and the Moonshine paper names LibriSpeech among its open training datasets [12]. (ii) *In-domain versus zero-shot evaluation:* the Whisper authors evaluate on LibriSpeech without its training split [23], so the table compares models that have seen the benchmark's training split with models that have not. (iii) *Test-set leakage:* that a test utterance itself appeared in any training corpus. We have no evidence of (iii) for any engine — LibriSpeech's own partitions are designed to be speaker-disjoint [21] — and the Whisper paper's transcript-level de-duplication is reported only against TED-LIUM 3, so it neither shows nor rules out LibriVox-derived audio in Whisper's web-scraped corpus. What the table supports is (i) and (ii): comparing an engine trained on the benchmark's training split with zero-shot ones measures the benchmark at least as much as it measures the engines, and the gap is not evidence that one architecture is better than the other.

What the row *does* support is narrower and still useful: a practitioner choosing an engine by its LibriSpeech number is reading a figure that is partly a property of who trained on LibriSpeech. Establishing a capability ranking on these CPU configurations needs a corpus held out from all three engines' disclosed training data, which we did not have; it is the first item of future work (§15). One trade-off is independent of the confound and worth naming regardless: as run here through onnx-asr, the transducer exposes no decode-time text prompt of the kind Whisper uses for vocabulary biasing, so a system that relies on prompt biasing for user-specific terms would lose that mechanism if it switched.

The fastest configurations are not the smallest Whisper ones. Moonshine [12] `base` reaches 43.1× real time at 8.04% — more accurate than `tiny.en` (11.77%) and `base.en` (9.46%), and faster than both. For a latency-bound path such as a live rolling transcript, this is the relevant corner of the frontier, and it is not occupied by the checkpoint a size-ordered list would suggest.

Threats. These are 200 stratified utterances of one corpus, one host and one decode per row. They should not be read against published full-set figures, which use different precision and hardware, and they characterize read speech rather than a microphone in a room; the absolute values are conditioned on everything in the provenance block. The claim is the comparison between rows *sharing a value in the training-split column* — within the Whisper family all five checkpoints are zero-shot here, and the four English-only ones have stable point estimates (some of which are not

resolvably different, as above); `large-v3`, as above, cannot be ranked from one decode. Across that column the table reports what a practitioner would see on this benchmark, not which engine is better.

5 The Same Audio Decodes to Different Text, and the Variation Is in Insertions

The width of `large-v3`'s interval in Table 2 prompted the obvious check: decode the same audio again. We ran 5 repeats of the same 200 *test-other* utterances under each of several decode configurations, storing a SHA-256 of each run's concatenated hypotheses — because two runs that trade one insertion for another score the same WER and are not the same text — and the ids of the utterances that differed.

5.1 The measurement

Table 3: 5 repeats per arm, 200 *test-other* utterances, `large-v3` int8 on CPU. "Distinct outputs" counts distinct hypothesis hashes across the 5 repeats. Aggregate substitution and deletion counts are identical in every run of every arm.

Arm	decode setting	WER (%)	distinct	ins.	subs./del.
`baseline`	faster-whisper defaults	4.84–6.21	**5 of 5**	78–129	87 / 15
`greedy`	`temperature=0.0`	15.26	1	466	87 / 15
`greedy_no_context`	both of the above	3.82	1	40	87 / 15
`no_context`	`condition_on_prev.=False`	**3.82**	1	40	87 / 15

Table 3 reports the result, and the striking column is the last one. Across every run of every arm the aggregate substitution count is 87 and the deletion count is 15. Not approximately: identically, and the same two figures appear in all 10 default-configuration decodes of Table 2's configuration, including the engine-matrix run and the four instability-probe runs. In these aggregates, everything the decode settings change on this checkpoint and split is added text. That is a property of this sample, not a law: on `base.en`/*test-clean* the arms differ by a few substitutions and deletions (from 141/16 to 146/19), and only within the one irreproducible arm do the substitution and deletion counts stay fixed while insertions move.

Two limits on that statement. The archive stores aggregate counts, not per-utterance hypotheses or per-utterance substitution/deletion/insertion splits, so we cannot show that no individual utterance traded a substitution for a deletion that another utterance reversed; exact equality across every run makes such compensation implausible, but that is an inference, not an alignment-level result. What the archive does show is how concentrated the variation is: across the 5 baseline repeats only 2 of 200 utterances ever differ, and they are the same two utterances whose `greedy`-arm transcripts run away (223, 104 errors against 29, 23 reference words, against 4, 1 errors in the baseline median run).

A design concern that follows, which we did not test with users, is that an inserted clause can read as fluent text and so be harder to notice than a dropped word — although a deletion can also be overlooked. This is related to the harms Koenecke et al. documented at the level of outcomes [15].

On this checkpoint and split the shipping default configuration is also the only arm that is irreproducible: 5 distinct transcripts in 5 decodes of identical input, WER ranging 4.84–6.21%, insertions ranging 78–129.

5.2 The mechanism

The arms differ in one keyword at a time, which localises the cause. Disabling the temperature fallback while leaving the decoder conditioned on its own previously emitted text (`greedy`) produces 466 insertions; additionally dropping the conditioning produces 40. In this sample, then, conditioning on previously emitted text is what the runaway insertions of the greedy arm require — they vanish when it is off — and they are concentrated: three utterances account for 393 of that arm's 568 errors. The temperature fallback appears to act as a rescue: with it on (the baseline) the same conditioning yields 78–129 insertions rather than 466. We did not count fallback rejections for `large-v3` on this split (they were counted only on `base.en`, and on a 40-utterance *test-clean* subset of `large-v3` where the fallback never fired), so the rescue is an interpretation of the arm contrast rather than an observed event. The rescue works by *sampling*, which is the likely reason the baseline is the only arm that cannot be reproduced: the two arms without sampled rescue (temperature 0) each return one hash.

The last two rows of Table 3 return the *same bytes*. With conditioning off, leaving the fallback on or off makes no difference to the output of this sample, so the safety net neither helps nor hurts reproducibility there. That is consistent with the fallback not changing the output; it is not proof that a sampled step never ran, as the next paragraph shows. `greedy_no_context` moves both knobs at once, and only the arm that moves one of them is the setting a system would actually ship.

Removing the cause is also cheaper in total decode time: 700 s per decode against the baseline's 829 s. We did not instrument why; fallback re-decodes avoided are the natural candidate, and the two arms were run at different times and loads, so this difference is indicative only. The median RTF is indistinguishable across arms, so a median hides the saving and a total shows it — itself a small lesson about which statistic to publish.

One inference that this probe *retired* is worth recording, because it is the kind that reads as obvious. On `base.en` the default and the fallback-disabled arms produce the same hash, which looks like proof that the fallback never fires. Counting faster-whisper's own rejection log instead shows it fires 4–8 times on that same checkpoint, across repeats that all return the same bytes (40 *test-clean* utterances; the rejection log is counted by a separate probe from the one behind Table 3). When every rung of the temperature ladder is rejected, the ladder ends by returning the best average-logprob result it saw — which can be the temperature-0 decode it started from. *Equal outputs are not evidence that a sampled step did not run.*

We also tested and rejected the natural follow-up hypothesis, that CTranslate2's multi-threaded int8 reduction order was responsible, since int8 accumulation makes thread completion order visible in the low bits. Across 28 decodes of 40 utterances at two `cpu_threads` settings, *every* hypothesis was bit-identical (1 distinct hash, WER 2.75% throughout) while the rejection count still moved — and it moved single-threaded too, which the hypothesis forbids. Splitting the count by temperature says what does move: the greedy rung at 0.0 is rejected exactly 3 times in every run, and only the sampled rungs above it vary (1–5). An utterance rejected greedily is re-decoded with sampling, so how far up the ladder it climbs is free to differ between runs while the text it finally returns does not. Pinning threads costs 2.05–2.19× wall-clock and buys nothing.

5.3 The sign of the effect changes with checkpoint size

Everything above is `large-v3`, which is not what a default install runs. Table 4 repeats the two-arm comparison across the checkpoint ladder on the same 200 *test-other* utterances.

The point estimates move monotonically with checkpoint size across the four checkpoints and reach exactly zero at `medium.en`, where both arms return the same hash across all ten decodes —

Table 4: Effect of disabling conditioning on previously emitted text, across checkpoints, 200 *test-other* utterances, 5 decodes per arm. Positive "effect" means disabling it *hurts*. The last column is the share of utterances taking more than one decode pass — the only ones a prompt can reach at all. † Every cell but this one is a single value reproduced by all 5 decodes; `large-v3` with conditioning on is *not* reproducible (Table 3) and the figure shown is the mean of a 4.84–6.21% range, so its effect inherits that spread.

Checkpoint	conditioning on	off	effect	multi-pass utts.
`base.en`	9.46	9.81	+0.35	16 / 200
`small.en`	5.59	5.7	+0.11	3 / 200
`medium.en`	5.51	5.51	0.0	1 / 200
`large-v3`†	5.52†	3.82	-1.7	—

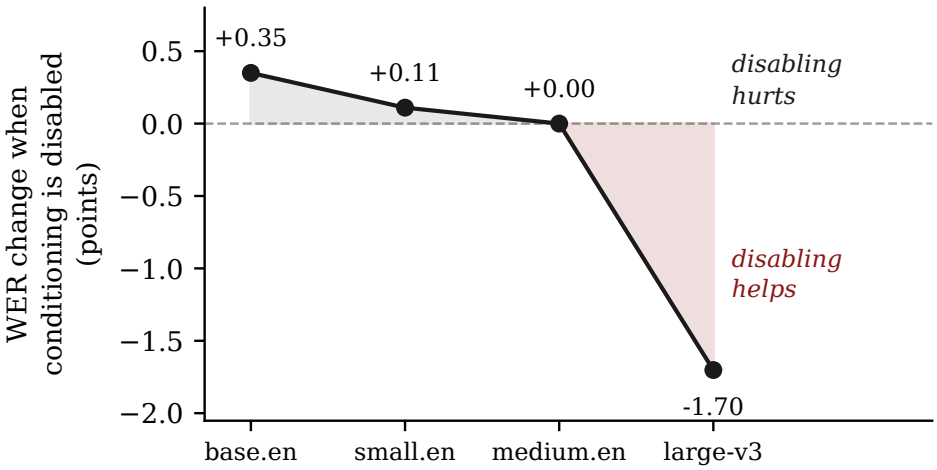


Figure 1: The point estimate of the conditioning effect changes sign with checkpoint size and passes through exactly zero at `medium.en`. Above the axis, disabling conditioning costs accuracy; below it, disabling gains. The `large-v3` point is measured against the mean of a non-reproducible arm and is not resolved when paired per utterance (Table 4, §6).

the flag did not change a single token in 200 utterances. Below `medium.en`, conditioning *helps* in these reproducible runs: turning it off costs 0.35 points on `base.en` and 0.11 on `small.en`, which on a corpus of 3721 reference words is of the order of a dozen and a handful of words; we ran no paired test for these two. At `large-v3` the sign of the point estimate flips: relative to the five-repeat mean, turning conditioning off *gains* 1.7 points, a gain that §6 shows is not resolved when paired per utterance. Figure 1 shows the pattern.

Reproducibility does not follow the setting either. The arm that is bit-identical five times out of five on `large-v3` is the one arm that is *not* reproducible on `base.en`/*test-clean* (5 distinct outputs across 5 decodes, WER 4.24–4.28%). On *test-clean* the shipping default is also the best arm, by a wide margin: 4.01% against 10.33% for the greedy arm, which emits 325 insertions against 30. *Neither the direction of the effect nor the reproducibility is a property of the setting alone.* A recommendation derived from one checkpoint, or one split, would have misled for the other cells we tested.

5.4 It reaches ordinary dictation, not only long files

The parameter is read in exactly two places in faster-whisper 1.2.1, both after a window is decoded and both only affecting the next one, which invites the inference that a single-pass decode of a short push-to-talk burst can never be affected — making all of this a concern for file import and meetings only.

That inference is false for the tested checkpoints. The decoder's seek position advances to the model's last emitted timestamp, not by a whole window, so a model that closes its final segment early leaves the remainder for another pass. Counting passes directly: 16 of 200 *test-other* utterances (8.0%) take a second pass on `base.en`, and on *test-clean* 8 of 40 (20.0%, longest clip 27.18 s) do. These are LibriSpeech utterances (median 5.88 s on *test-other*), somewhat longer than a typical push-to-talk burst, so the share for real dictation is not measured here. With conditioning on, *every one* of those later passes is handed the first pass's text (16 and 8 respectively); with it off, none is. The flag acts on audio of dictation-like length.

The share falls with checkpoint size — 8.0% on `base.en`, 1.5% on `small.en`, 0.5% on `medium.en` — which explains the shape of Table 4 but not all of it: if delivery frequency were the whole story, `medium.en` would show a small effect rather than none. Its single multi-pass utterance *was* handed previous text and the output was identical anyway. Two things shrink together: how often the prompt is delivered, and how much the model is moved when it is.

6 A Corpus Mean Can Be a False Description of What a User Gets

Section 5 supports a tempting headline: 3.82% against the five-repeat mean of 5.52%, a 1.7-point win, from changing one keyword. That sentence is true arithmetic and a false claim about what a user experiences, and we report it here because it is what this study would have published had the arms not been re-scored per utterance.

A corpus WER is a weighted mean, and a mean over 200 utterances is moved a long way by a handful of badly wrong clips. Scoring every utterance against a paired baseline gives a different picture. The pairing uses the median-WER run of 3 separate baseline decodes taken for this probe (WER 6.074, 4.784, 4.864%), whose corpus WER is 4.864%; against that baseline the paired gain is 1.05 points, not 1.7, because the baseline itself moves by more than a point between draws:

	ΔWER	95% CI	better / worse / unchanged	sign test p
conditioning off	-1.048	$[-2.589, 0.161]$	4 / 8 / 188	0.3877

The interval crosses zero, *more* utterances got worse (8) than better (4), the sign test does not reject, and 95.7% of the errors removed (45 of 47) come from three utterances — 38.3% from a single one. The denominator is the *gross* reduction, summed over the utterances that improved: the 8 utterances that got worse add 8 errors, so the net change is 39 fewer errors, and the three clips account for more than the whole of it.

Two dependence checks, since the interval above resamples utterances as if they were independent and conditions on one baseline draw. First, the 200 utterances come from 33 speakers; resampling speakers, with each speaker's utterances kept together, gives $[-2.56, 0.14]$ for the same paired difference, so the interval still spans zero. Second, the pairing baseline is the median of 3 stochastic draws, and the other two draws would give a different point estimate: measured against each draw's corpus WER, the change from disabling conditioning is -2.26 to -0.97 points. The archive keeps

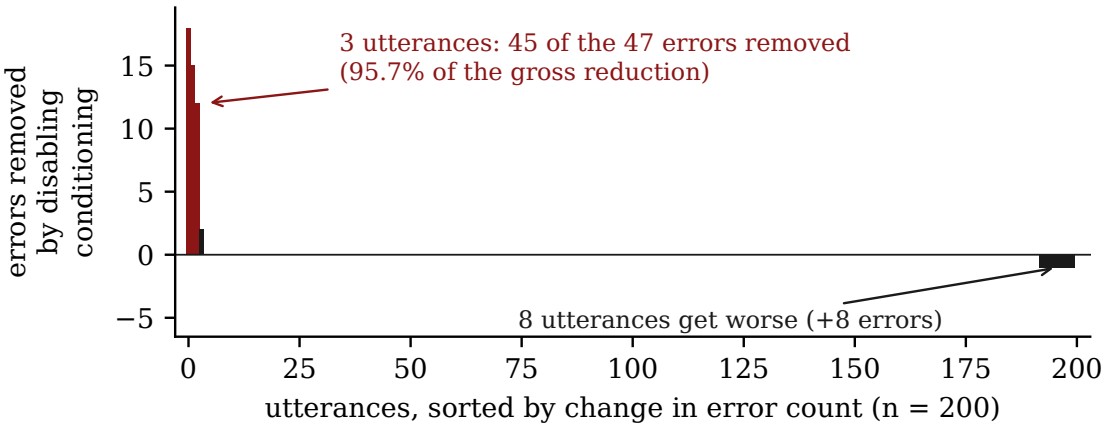


Figure 2: Per-utterance change in error count when conditioning is disabled (`large-v3`, 200 *test-other* utterances, paired against the median-WER run of 3 baseline decodes). Three spikes and a flat line: this is what a −1.048-point corpus mean is made of.

per-utterance rows only for the median draw, so those two comparisons have point estimates and no intervals, and the uncertainty from the decoding randomness of the baseline is not folded into the interval above. The conclusion we draw — the paired benefit is not resolved — stands; the size of the benefit is sensitive to which baseline draw is used.

Figure 2 is the distribution the mean summarised away.

The honest argument is therefore not about the mean. It is about the tail — and the tail has to be sized correctly, because the two arms fail at very different magnitudes. In the *shipping* configuration the three utterances carrying the gross reduction have 20, 17, 14 errors against 28, 23, 40 reference words, i.e. transcripts that are 71, 74, 35% wrong. Plausibly that is a burst a user would need to delete and redictate rather than a one-point WER event — a design hypothesis, not a measured user effect — and it is not, however, hundreds of inserted words. The catastrophic case lives in the `greedy` arm, where a single utterance produces 223 errors against 29 reference words — the same clip the shipping default handles with 4. So on this clip the temperature fallback *is* earning its place: it converts a catastrophic failure into a merely bad one, at the price of making the output irreproducible (§5).

Disabling conditioning removes the bad cases too, and costs a small, statistically unresolved amount on ordinary speech (8 utterances got worse, each by one error). *That* is a reason to consider changing a setting on this checkpoint, subject to this being one sample of 200 utterances and one library version. A WER win is not.

We draw a general recommendation from this, as a design principle rather than a measured user effect. For an interactive on-device system, a corpus mean can be a misleading summary statistic for a decode-configuration decision if the user's experience is dominated by rare catastrophic outputs rather than by average error. Reporting the per-item distribution and the concentration of the error reduction alongside the mean costs nothing and, in our case, changed the conclusion.

7 Accuracy Spread Across CI Runners Shrank as Checkpoints Grew, in the Three Compared; Timing Does Not

The companion paper's largest threat to validity was that every number in it came from one Linux laptop. We dispatched the same harness to four GitHub-hosted runners spanning three operating systems and two ISA families, x86_64 and arm64 (Tables 5 and 6), on a nominally identical, deterministically selected set of 60 utterances with the same checkpoints and the same CTranslate2 version (4.8.1; the wheel build differs per platform). The runners differ in more than the instruction set — CPU model and generation, operating system, Python version (3.12.10–3.14.6) and load — so the design is a robustness check across environments, not a controlled test of the ISA.

Table 5: The same code and a nominally identical, deterministically selected set of 60 utterances (the CI artifacts store neither ids nor a digest) on 4 runner environments: AMD EPYC 9V74 80-Core Processor (Linux x86_64), Neoverse-N2 (Linux arm64), Apple M1 (Virtual) (macOS arm64) and AMD EPYC 7763 64-Core Processor (Windows x86_64). WER in %. "Spread" is max − min across the row.

Checkpoint	Linux x86_64	Linux arm64	macOS arm64	Windows x86_64	Spread
`tiny.en`	3.39	3.6	3.74	3.88	0.49
`base.en`	3.32	3.32	3.25	3.39	0.14
`small.en`	2.05	2.05	2.05	2.05	0.0

Table 6: What differs between the runners besides the instruction set (generated from the provenance blocks; "n/r" = load average not recorded on Windows).

Runner	CPU	OS / Python	vCPUs	load
Linux x86_64	EPYC 9V74	Ubuntu 24.04 / 3.12.3	4	0.44
Linux arm64	Neoverse-N2	Ubuntu 24.04 / 3.12.3	4	0.52
macOS arm64	Apple M1 (virt.)	macOS 26 / 3.14.6	3	30.44
Windows x86_64	EPYC 7763	Windows Server 2025 / 3.12.10	4	n/r

In the three checkpoints compared the spread narrows as the checkpoint grows (Figure 3); three points are an observation, not a law: 0.49 points on `tiny.en`, 0.14 on `base.en`, and 0.0 on `small.en`, where all four runners return 2.05% — the same number to the last reported digit. The ISA family does not separate the `tiny.en` results: the largest spread (0.49) is between the two *x86_64* runners, which are different CPU products (Table 6), against 0.14 between the two arm64 runners.

One plausible explanation is environment-dependent numerical behaviour of int8 kernels, whose accumulation order can differ with the CPU and library build; a larger model's margins between candidate tokens may be wide enough that low-bit differences no longer flip a decision. This experiment cannot establish that mechanism: the runners differ in CPU model, operating system, Python version and kernel build, and each decoded the corpus once. A related hypothesis — that thread-completion order in the multi-threaded int8 reduction explains run-to-run variation — was tested and rejected on one host in §5.

Three cautions, and the third is one our own §5 forces. First, timing does *not* transfer: the macOS runner reported a one-minute load average of 30.44 on three logical CPUs, so its RTF figures describe a contended host and belong in no comparison — which is why this table reports

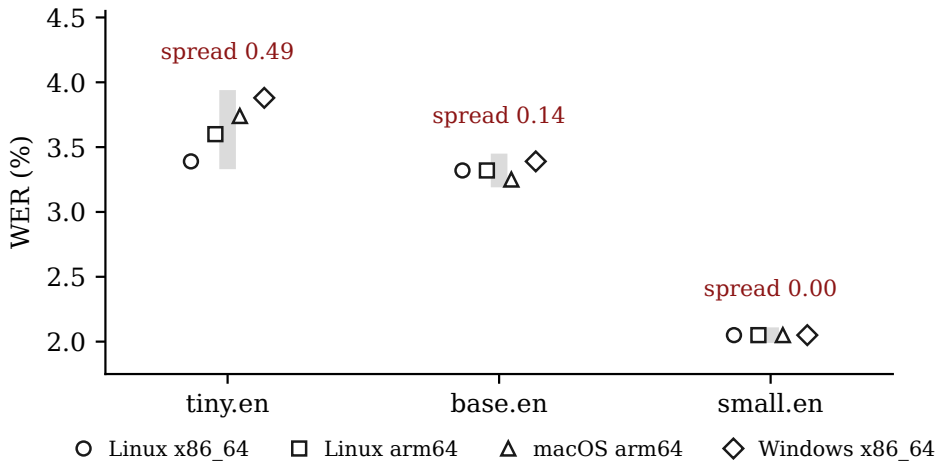


Figure 3: A nominally identical set of 60 utterances decoded by the same code on four runner environments (two ISA families). The vertical bar is the spread across runners; it collapses to zero by `small.en`. Accuracy only — the timing figures are not comparable across these hosts (§7).

accuracy only. Second, this is 60 utterances on shared CI hardware of unknown neighbours, not the 200-utterance figures of §4.

Third, **each runner decoded the corpus once**, so this table cannot by itself separate an instruction-set effect from the run-to-run variance §5 documents. That separation has to come from repeats, and we ran them. On the shipping decode configuration `base.en`, `small.en` and `medium.en` each return a single hypothesis hash across 5 repeats on one host. `tiny.en` — the row carrying the largest spread, and the one checkpoint the rest of this paper never exercised — was left untested until this section needed it: 5 repeats of 60 *test-clean* utterances return 1 distinct hash and WER 3.67% every time. All four checkpoints were bit-reproducible in these within-host repeats on one x86_64 laptop, which makes stochastic decoding on that host an unlikely explanation for the observed CI spread. The repeats do not exclude within-runner variability on the four CI hosts, each of which was measured only once, or differences in the utterances actually selected.

Two limits on that repeat, since it is the load-bearing one. It ran on the laptop rather than on any of the four runners, so it establishes reproducibility on an x86_64 host and not separately on `aarch64`; and the CI artifacts record no corpus digest or utterance ids, so we cannot prove they scored the identical 60 utterances (our subset's WER, 3.67%, falls inside their 3.39–3.88% range, which is consistent with but not proof of the same selection). What the repeat makes unlikely is run-to-run variance as the explanation on that host. Separating that explanation from environmental effects (CPU, operating system, Python version, kernel build) would need repeats on each CI runner with pinned utterance identifiers or a corpus digest, and attributing the spread to any one factor would need a controlled design that varies one factor at a time.

A fifth runner is missing, and the reason is itself a finding for §12: the Intel-macOS leg could not install, because a pinned ONNX runtime version publishes wheels for Apple silicon only. The workflow's "continue on error" setting turned that into a silent absence rather than a red job — a matrix row that looked measured and never was.

8 Decode-Parameter Conclusions Did Not Transfer Between the Cells Tested

Beam width is the parameter an on-device system is most likely to tune, because it trades latency for accuracy directly. Table 7 sweeps it on two checkpoints and two splits.

Table 7: WER (%) against beam width b, 200 utterances per cell. The upper block is the checkpoint the user dictates with; the lower is the checkpoint the latency governor switches to under load.

Grid	Split	b=1	b=2	b=3	b=5	b=8
`base.en` (dictation)	test-clean	4.39	4.01	4.07	4.01	4.01
`base.en` (dictation)	test-other	10.56	9.49	9.51	9.46	9.84
`tiny.en` (governor)	test-clean	5.53	5.12	—	4.95	—
`tiny.en` (governor)	test-other	12.42	12.04	—	11.82	—

On the dictation checkpoint, greedy decoding is worse than beam 2 in both splits — paired bootstrap $p = 0.0026$ on *test-other* (a 1.075-point difference, 95% interval $[0.263, 2.328]$) and $p = 0.0242$ on *test-clean* (0.386 points, $[0.061, 0.748]$) — but these are two of a family of 5 comparisons per split, and after Bonferroni correction only the *test-other* result stands (0.013 against 0.12 on *test-clean*). Beyond beam 2 nothing is resolved: against beam 2 as the baseline, beam 5 gives $p = 0.964$ and beam 8 gives $p = 0.3994$. So the operating point is beam 2, and everything above it is latency spent for no measurable accuracy in this sample — an absence of a resolvable difference, not evidence of equality.

The transfer failure is the interesting part. YazSes has a latency governor that, under CPU load, switches to a smaller checkpoint *and* narrows the beam. Scoring beam 1 on the dictation checkpoint says nothing about that policy, because the policy never decodes that combination. On the checkpoint the governor actually runs, the beam-1-versus-2 comparison is not resolved: $p = 0.4118$ on *test-other* and $p = 0.0986$ on *test-clean* (beam 5 against beam 1 reaches a nominal $p = 0.0234$ on *test-clean* and 0.264 on *test-other*, uncorrected). With 200 utterances we cannot say whether narrowing the beam costs accuracy on the checkpoint it is applied to; a cost that is measurable on one checkpoint is simply not measured on the other.

The general form: *a decode-parameter measurement should be treated as valid only for the (checkpoint, split, quantization) cell it was taken in until it is replicated elsewhere.* The `small.en` beam-1-versus-5 comparison is $p = 0.053$ on the hard split and $p = 0.1474$ with the opposite sign on the clean one — neither resolved, so this illustrates uncertainty rather than a reversal — and the conditioning effect differs in sign across checkpoints in §5. An adaptive policy that switches models at runtime must be measured in the configuration it switches *to*, and we suspect this is a common omission because the tuning experiment is naturally run once, on the default model.

9 Two Negative Results

9.1 The silence lead-in does not survive multiplicity correction

Push-to-talk systems commonly prepend a short silence before the audio handed to the decoder, on the theory that Whisper clips the first word without it. YazSes does this, with a configurable duration. We measured first-word accuracy across a grid of lead-in durations (0, 100, 300, 600, 1000 ms) crossed with deliberately clipped speech onsets (40, 120, 240 ms, simulating a late keypress),

200 utterances, 2 repeats, scored with an exact McNemar test per cell against the zero-lead-in baseline.

Of 32 comparisons, 4 reach $p < 0.05$. Reducing to 16 distinct comparisons and applying a Bonferroni correction [8] at $\alpha = 0.00313$, **none survives**: the smallest p-value over the whole family is 0.0113, approximately 3.6 times the corrected threshold. The 2 distinct cells that clear the uncorrected $\alpha = 0.05$ (`clipped cut=120 lead=1000`, $p = 0.0113$; `clipped cut=120 lead=600`, $p = 0.0336$) are both in the *clipped*-onset condition and both point the *opposite* way to the design rationale — a long lead-in doing worse than none — but neither is significant once the family is accounted for.

We report this rather than quietly retaining the feature. The honest reading is that on this corpus the lead-in is not measurably doing what it was added to do, and that the grid was large enough that some cells would clear $p < 0.05$ by construction. We chose the first-word metric rather than utterance WER because a repeat of the clipped grid reproduced all 9 first-word counts exactly while utterance WER moved by up to 1.0 point — the coarser metric was the stable one. That stability is visible in the test itself: the McNemar test is run per repeat rather than pooled, and 12 of the 16 distinct comparisons return the *same* p-value in both, because the underlying first-word counts did not move. Where a comparison differs between repeats we report the larger p, so the family is judged on its weaker evidence rather than its luckier repeat.

9.2 Streaming decode did not pay for short bursts in this measurement

LocalAgreement streaming [17] emits a prefix once successive decodes agree. It is designed for unsegmented long-form speech; the question here is whether it helps a push-to-talk burst, where the user holds a key for a few seconds and wants text to appear. We fed 15 utterances (median 10.44 s) at real-time pace and compared against the batch decode it would replace.

This is a timing measurement, so its host matters and Table 1 names it: the laptop, at a one-minute load average of 24.13. By the standard we applied to §7, the *absolute* latencies below therefore describe a contended machine and should not be quoted alone. Both arms were fed the same utterances in the same run on that host, which helps, but contemporaneous runs do not remove time-varying contention or scheduling effects, so the comparison describes this contended run and no more. A repeat on an idle host, and at varied load, is needed before it is generalised; it would have to close a 1.9× gap to change the sign of the result.

On the shipping checkpoint the answer, for this sample of 15 utterances, is negative. Median text visible at key-release: 0.0%. Utterances with no partial hypothesis at all before release: 12 of 15. And the final text arrives *later* than batch — 7.612 s against 4.024 s median from end-of-speech. On the smallest checkpoint streaming does show something (59.3% visible at release, 0 utterances with no partial) — but at that checkpoint's accuracy (Table 2: 11.77% against 9.46% for the default, on a different corpus), and its final text is *still* later than batch (2.371 s against 1.377 s).

A likely reason is structural rather than an implementation defect (we did not test other regimes). LocalAgreement needs at least two decodes of an overlapping window before it may emit anything. On a CPU where one decode of the burst costs a meaningful fraction of the burst's own duration, the repeated decodes do not finish inside the hold. Streaming buys perceived latency when decoding is much faster than real time and the utterance is long; a push-to-talk burst on CPU is the opposite regime on both counts. We therefore keep the batch path as the default and report the streaming path as measured-and-rejected rather than as a feature.

Table 8: Diarization on 16 AMI test meetings (8.5 scored hours), sherpa-onnx with powerset segmentation [22] and ERes2Net embeddings [6], at clustering threshold 1.2. All figures in %, except the speaker-count rows (in speakers). Signed error is hypothesised minus reference speakers, so an undercount offsets an overcount (12 recordings are over-split, 2 under-split, unpinned); the absolute row does not offset.

	Unpinned	Pinned to 4
DER, strict (unweighted mean)	26.71	29.42
DER, strict (time-weighted)	27.37	29.32
DER, 250 ms collar (unweighted mean)	19.93	23.39
DER, 250 ms collar (time-weighted)	20.51	23.35
Missed speech	8.66	8.66
False alarm	4.5	4.46
Speaker confusion	13.54	16.3
Mean signed speaker-count error	+2.06	+0.06
Mean absolute speaker-count error	2.44	0.06

10 Long-Form: Diarization on Real Meetings, and a Configuration Trap

10.1 The headline numbers, and why we report two

Table 8 reports diarization error rate over 16 AMI test meetings [5] (headset mix, 8.5 scored hours) at the shipped clustering threshold of 1.2.

We report both an unweighted per-recording mean (26.71%) and a time-weighted figure (27.37%) and the distinction is not cosmetic. The mean answers "how well does this do on a meeting", and a forty-minute meeting should not drown out three short ones. The time-weighted figure is the quantity `md-eval` computes [9] and the usual convention for published tables, so it is the one to compare against the literature — under the same scoring protocol (collar, overlap handling and reference annotations), which we did not match to any particular published system. On a corpus whose recordings differ in length by 3.5× they do not have to agree, and neither is a correction of the other. Reading a per-recording mean as though it were a corpus DER compares two different quantities — a small point, but one we found easy to get wrong. Neither figure is held out: the clustering threshold was selected from a sweep on four of these meetings and confirmed on the full set (see Threats to Validity), so Table 8 is in-sample for that parameter.

10.2 Pinning the speaker count silently voids the tuned threshold

Interactive tools commonly let the user say how many speakers there are, and it is natural to assume this can only help. In the harness the count is passed as `-max-speakers 4`; 1 of the 16 meetings has three speakers, which is why the count error is not exactly zero. It does help the one thing it directly controls: the mean signed speaker-count error falls from +2.06 to +0.06 (mean absolute error from 2.44 to 0.06). But the mean DER is *higher* (26.71% → 29.42%, +2.71 points), almost entirely through speaker confusion (13.54% → 16.3%) while missed speech and false alarm are essentially unchanged — a difference the next subsection shows is not resolved.

The mechanism is a silent interaction between two settings. The clustering backend uses its distance threshold *only when the cluster count is unset*; pinning the count makes the threshold inert. That threshold is precisely the setting we had tuned, and the tuning is worth 48.5 DER points: at

its previous default of 0.5 this corpus scores 75.21% against 26.71% at 1.2. None of that reaches a user who passes a speaker count. We verified this rather than inferring it, and the result is an exact equality: both pinned runs score 29.42% — at the tuned threshold 1.2 and at the old 0.5 alike — bit-identical on all 16 recordings, and a regression test now asserts that equality so a future version that makes the threshold matter fails a test instead of misleading someone.

10.3 And the difference it appears to make is not resolvable

Having found the trap, the tempting next sentence is that pinning costs 2.71 DER points. Paired over the 16 recordings it does not survive: 7 recordings better, 7 worse, 2 unchanged, exact sign test $p = 1.0$, with both bootstrap intervals crossing zero (mean change +2.71, interval $[-1.64, 7.6]$; time-weighted +1.96, interval $[-3.39, 7.69]$). The two largest increases (+28.72 and +15.48 points) sum to more than the whole net change over the sixteen recordings (+43.4). This does not show the two settings are equivalent; it shows this sample cannot resolve their difference.

That paired analysis treats the sixteen recordings as independent, and they are not: they are 4 series of four. Averaged by series, the change from pinning is −2.96 points on EN2002, +3.41 on ES2004, −0.55 on IS1009 and +10.95 on TS3003, against the pooled +2.71, so the pooled figure averages effects of both signs. Resampling the four series gives a 95% interval of $[-1.8, 8.1]$ and leaving out one series at a time moves the mean between −0.03 and +4.60 points. With four clusters those intervals are very imprecise, and we offer them as a reason not to read the sixteen recordings as sixteen independent groups, not as a repair of the small sample. Three of the four series are scenario-based, so these data say little about naturally occurring meetings.

This is the same lesson as §6 arriving in a different domain, and we state it as a rule: *report the per-item rows, and test paired, before quoting any difference between two long-form configurations.* A corpus of sixteen recordings is small enough that one meeting moves the mean by more than the effect being claimed.

10.4 An attractive repair that cannot work

The dominant error above is over-splitting: one speaker's turns land in several clusters. The obvious post-hoc repair is to merge clusters whose embedding centroids are close. We measured whether that is possible at all, by computing every cluster-pair centroid cosine and labelling each pair by whether it is really one speaker (45 same-speaker pairs, 210 different-speaker pairs, using the same embedding model the diarizer clusters with).

The distributions overlap almost completely. Same-speaker cosines span $[-0.1421, 0.9766]$ and different-speaker cosines span $[-0.0432, 0.8273]$. Sweeping a merge threshold across the whole range, the best operating point observed on this sample (in-sample, not a population optimum) is recall 55.6% at precision 65.8% ($F_1 = 60.3$) — meaning that repairing roughly half the over-splits requires wrongly merging two genuinely different speakers about a third of the time. For a meeting transcript, a wrong merge is worse than an over-split: an over-split shows the reader two labels for one person, which can be seen and fixed, whereas a wrong merge attributes one person's words to another, which is harder to notice (a design judgement, not tested with users).

We report this as a negative result with a purpose. Centroid-cosine merging is an obvious thing to try and the failure is not visible from a fragment-level test; it is visible only from labelled cluster *pairs*, which is a measurement most diarization evaluations do not produce.

10.5 A guard on attribution, and its firing rate

Rather than repair the clustering, the shipping system warns when the attribution is implausible. The guard's value depends entirely on how rarely it fires, because a guard that interrupts on healthy input teaches the user to dismiss it. On 16 AMI meetings — of which 12 are genuinely over-split — it fires 1 time and is right 1 time, with 0 false alarms: it catches 1 of the 12 over-split meetings (observed sensitivity 1/12) and misses the rest. On 15 VoxConverse recordings [7] at one clustering threshold it fires 6 times with 0 false alarms, and at another it fires 4 times with 0.

Two honest qualifications. The duration-scaled formulation was chosen *after* seeing that a flat-threshold formulation produced 3 false alarms on the harder VoxConverse setting, so its zero false-alarm record on these corpora is partly a fitted result rather than a held-out one; a zero false-alarm count on a few corpora is not a specificity guarantee. And the guard's value here is that it rarely interrupts, not that it finds over-splitting: on these data it is a conservative alarm with low observed sensitivity, and it is silent on everything other than over-splitting — notably the failure in §11, which is the more dangerous one.

11 A Silent Long-Form Decode Collapse, and a Cross-Decode Consistency Check

This section reports a single observed failure rather than a matrix, because the failure is qualitative and its consequence — the loss of the recording — was irreversible. Unlike the other sections it rests on no results artifact: the meeting is private, and the figures below are transcribed from the shipped source's record of the incident (release notes for version 2.32.0, the `meeting/quality.py` module and its tests).

11.1 The failure

On 2026-08-26 a 41-minute four-person meeting was recorded and transcribed by the shipping system on a maintainer's machine. The batch pass returned normally. The resulting transcript was 284 words, dominated by the phrase "Hello, hello, hello" repeated 93 times (279 words; the remaining 5 are not itemised in the archived record). The meeting metadata recorded `status: "done"` and `capture: "ok"`. Because the post-pass had consumed the recording successfully — by the only definition of success the system had — the audio was deleted, in accordance with a privacy-motivated default that does not retain recordings. The rolling live transcript, kept only as a crash-recovery format, held 4,553 words of live-decoded text, and nothing in the product rendered it, listed it, or mentioned it for a meeting that had finished.

Every existing guard passed, and each was right to. One asks whether audio was *heard*; it was. Another, an attribution plausibility guard, looks for implausible clustering, particularly over-splitting; the failed batch transcript had a single hypothesized speaker cluster, so it did not fire. That validated neither its speaker labels nor its words: the meeting had four people and the transcript was unusable. Neither asks whether the *words are real*. Nothing did, because until this failure "the post-pass returned" and "the meeting was transcribed" were treated as the same statement.

This is the long-form form of the degeneration failure [11] that §5 isolated at the utterance level, and it demonstrates the paper's thesis in its sharpest form: the output was syntactically perfect, structurally complete, carried timestamps and word alignment, and raised nothing.

11.2 A consistency check between two decodes

We added a gate that judges a finished transcript from four statistics — top n-gram share, distinct n-gram ratio, longest back-to-back repeat of a phrase, and words per minute over a minimum duration — with thresholds (top-trigram share ≥ 0.2, distinct-trigram ratio ≤ 0.35, a repeat run of at least 12) chosen after inspecting the metrics of the five real stored meetings on the machine where the failure happened. That makes its record a fitted, in-sample one, not a held-out validation: recall 1/1 and false alarms 0/4 on that set. Numerical live and batch word counts are reported for only two of the five meetings (the collapsed one and one healthy one). Per-meeting metrics are archived in the shipped source for three of the five (the collapsed meeting and two healthy ones); for the other two healthy meetings only the verdict is recorded, in the release notes. On the archived three, the strongest repetition signal separates the collapsed decode from the worst healthy one by roughly two orders of magnitude (top-trigram share 0.9681 against 0.0101), and the second by roughly one (distinct-n-gram ratio 0.0355 against a worst healthy 0.821). Five meetings is a small set and we do not claim more from it than that the thresholds are not sitting against the one bad sample.

A defect in the first version of the repeat-run signal, found in review. The version of the gate assessed above counted a back-to-back repeat only when an overlapping trigram equalled its *immediate neighbour*. That sees the one collapse we observed, a single word said over and over, and is blind to a loop of two or more distinct words, whose overlapping trigrams rotate through several values so that the run never exceeds one. We reproduced this on the shipped code with a synthetic transcript, not a new decode: a healthy 250-word start followed by a three-word phrase repeated 16 times has a repeat run of 1, a top-trigram share of 0.054 and a distinct-trigram ratio of 0.855, so no threshold above fires and the transcript is called healthy. This is the late-onset case the run signal was added for. The whole-transcript ratios do catch such a loop once it dominates the text, but not while the rest of the meeting is intact. We have since extended the check to phrases of up to 8 words (the same transcript then has a run of 15 and is flagged), with regression tests. Status of that change at the time of writing: it is a single commit (`8785e334`) on a branch of the public repository (`fix/repeat-run-multiword-loops`); it is *not* merged into the default branch and is in no release, so the detector that ships, and that the rest of this section assesses, is still the first version. The extension is covered by synthetic tests only: no multi-word collapse has been observed in the wild, so we have no evidence about its false-alarm rate on real long-form speech, and a looping unit longer than 8 words is still outside the run signal's reach.

A second signal compares two decodes of the same audio. It is dimensionless but not threshold-free. The system already decodes each meeting twice with different configurations — a rolling, incremental live pass during capture and a whole-file batch pass at stop. The ratio of their word counts is dimensionless, but we have not established whether it varies systematically with recording duration or speech content, and an alarm still needs a cut: we flag at $\geq 3\times$ with a minimum of 20 batch words, a value chosen by us and not calibrated on a population. On the collapsed meeting the live pass holds 4,553 words against the batch pass's 284, a ratio of 16.0×; on one healthy two-hour meeting the two agree to 1.008× (16,000 against 15,874 words). The two passes are not independent opinions in the statistical sense — they share the audio and any systematic blind spot — so their agreement is weaker evidence than their disagreement. The ratio does, however, give a cheap extra signal that a gate computed from the batch transcript alone lacks.

The response matters as much as the detection. The recovery design has four parts. The live transcript is rendered to a durable artifact *before* the batch pass runs (so a finalize that dies still leaves it), independently of that pass's eventual verdict. If the batch transcript is judged suspect,

the recording is retained regardless of the privacy default, the summarization pass is skipped (a summary of invented words is hard to audit), and the meeting is offered for re-processing even though it finished. The retention is a deliberate exception to the system's delete-after-transcription default, made for exactly the meetings whose transcript cannot be trusted. We describe it as an exception and do not claim more about it: this paper does not evaluate a retention time bound, encryption of the retained recording or a deletion timer. The verdict is written for every meeting, healthy or not, because a verdict is only interpretable next to the numbers it did *not* fire on.

What it does not catch. This gate detects *collapse* and *silence*. It does not detect *inaccuracy*: a fluent hallucination at a normal word rate passes every signal, including the cross-decode ratio if both decodes hallucinate similarly. Detecting a collapse is not detecting hallucination in general. We state this plainly because a quality gate whose limits are not published will be trusted past them.

Generalization. The specific thresholds are ours; the architectural claim is not. Any on-device pipeline that (a) produces a long-form transcript, (b) deletes its input on success, and (c) defines success as "the decoder returned" has this failure mode. One cheap addition is not a better statistic but a second decode — which a hybrid live/batch design already has.

12 Dependency Resolution Blocked Some Install Targets That No Accuracy Table Shows

A recurring assumption in on-device ML is that the binding constraint is the user's hardware. We did not test that assumption, and this section does not support or refute it: it reports a different, static failure that sits upstream of any hardware limit. We resolved the full dependency closure of each of 23 install targets — the 22 optional feature sets plus the base install — against 4 platform targets, 92 cells in total, classifying each failure by cause. The platform targets are the resolver's assumed floors (for example `manylinux_2_28` on Linux and `macosx_13_0` on macOS), not a survey of the operating systems users actually run.

84 of 92 resolve. 8 do not. The check is static dependency resolution, so it cannot observe runtime CPU or memory limits, and we did not assess those; none of the failures is a resolution failure for lack of any wheel at all (0 cells). 6 fail against the operating-system floor the resolver assumed: a dependency publishes wheels for the architecture, but built for a newer OS than that floor. Whether that makes the feature unavailable to a given user depends on that user's OS version, which this check does not know. Three of these cells are PySide6 on Linux, whose current wheels target `manylinux_2_34`, above the assumed floor, and three are `onnxruntime` on Apple silicon against `macosx_13_0`. 2 fail against a version floor on an abandoned architecture: a dependency's last release for Intel macOS predates the minimum version another dependency requires, so two requirements that are individually satisfiable are jointly unsatisfiable on that architecture. Those two are a defect in our own declared floors, not in upstream.

This has three consequences we did not anticipate. First, a feature's availability is a moving target set by upstream release policy rather than by anything in the project — a cell that resolves today can fail tomorrow with no local change. Second, the failure is invisible without a resolution matrix: it appears as an installation error on one user's machine, not as a test failure. Third, and most practically, the cost of enabling a feature is not the size of its own wheel: one speaker-embedding library is a few megabytes itself and 3,363 MB across 40 packages once resolved, so pricing a feature

by its direct dependency would call it cheap immediately before it fills the user's disk. We now report the marginal resolved closure size before installing anything.

We suggest that a dependency-resolution matrix belongs in the evaluation of any on-device ML system that ships optional features, alongside the accuracy tables. It is cheap to produce, it is a failure mode users can hit before any accuracy question arises, and it answers a question no benchmark does: on which machines does this feature exist at all? The finding generalizes to the extent that a project's packaging ecosystem resembles this one's — it is a property of Python wheels and upstream release policy, not of machine learning — so the numbers here are ours and the practice is what we are recommending.

13 Discussion

Pulling the findings together, six recommendations for builders of on-device speech systems are motivated by measurements in this paper, and a closing note on the trade-off that produced the worst failure in it.

A second decode is a cheap consistency signal, not a correctness test. Disagreement between two differently configured decodes of the same audio was a salient warning in the one documented collapse (§11), and disagreement between two decodes of the same audio *by the same configuration* exposed the run-to-run instability of §5. Only two meetings have numerical live and batch word counts, and we compared neither signal with any other, so we cannot say which is more informative or how either performs; their sensitivity and specificity are unmeasured. Neither needs a labelled corpus or an extra model (an alarm threshold is still needed). We do not claim a second decode is necessary: recognizers can also estimate confidence from a single pass [13], and we did not compare against those estimators. Nor is agreement evidence of correctness — two decodes that share the audio can share an error — and a disagreement says that something is unstable, not which decode is right. The 3× alarm threshold comes from two meetings and has not been tested on held-out data, and we did not measure the CPU, energy or latency cost of the extra decode. What the incident supports is narrower: a system that gates an irreversible action solely on a decoder's successful return has no evidence that the content is correct.

Report insertions separately. The run-to-run instability in §5 is not visible in the WER of any single decode and shows up in the insertion counts across repeats. Insertions include fabricated content, which we hypothesize — without having tested it — may be harder for a user to notice than a dropped or substituted word, although any of the three can be overlooked. For an interactive dictation system we suggest reporting them separately in addition to WER.

Measure in the configuration you ship, including the ones you switch to. Beam width (§8) and conditioning (§5) both differ in sign or significance across the checkpoints and splits tested. An adaptive system that switches models under load must be measured in the configuration it switches *to*, which is exactly the measurement a one-off tuning experiment skips.

Check what your baseline was trained on before you rank engines. The single most misleading number we produced was a clean, correctly measured, correctly bootstrapped WER comparison between models with different relationships to the evaluation corpus (§4). Nothing in the measurement pipeline could have caught it, because nothing was wrong with the measurement.

We expect practitioners to choose engines from published benchmark numbers at least as often as from their own, which would make this a standing hazard rather than one paper's mistake.

Publish the per-item distribution, not only the mean. Two of our apparent effects were not resolved when re-scored per item (§6, §10); both would have been published as real effects. The per-item rows cost nothing to store and are the only thing that distinguishes a broad improvement from three lucky clips.

Treat "the call returned" as no evidence of success. The most expensive failure in this study (§11) was a clean return. A pipeline that gates an irreversible action — deleting a recording, overwriting a prior result — on the absence of an exception is gating it on the one signal that had failed.

On the choice of privacy defaults. The deletion policy that lost the meeting is a good privacy default and we have not changed it; we changed what counts as consumption. That is the general shape of the trade-off in on-device systems: privacy defaults remove the safety net that server-side retention provides, so the verification has to move into the client rather than the default being weakened.

14 Threats to Validity

Corpora. Short-form results are 200-utterance speaker-stratified subsets of two LibriSpeech splits: English read audiobook speech, which is a floor rather than a guarantee for noisy microphones, accented speech or conversational prose. They should not be read against published full-set figures. Long-form results use 16 AMI test meetings on the headset mix, which is cleaner than a single table microphone; a far-field recording would score worse.

Training-split overlap. This is the sharpest limit on §4. Parakeet's and Moonshine's publishers state that LibriSpeech training data was used; Whisper is evaluated on LibriSpeech zero-shot [23], which excludes the training split but does not audit the web-scraped corpus. The cross-family rows of Table 2 are therefore *not* a capability ranking and we do not present them as one; we also do not claim test-set leakage for any engine. The within-Whisper comparison is unaffected by the confound, because all five checkpoints are zero-shot on this corpus. The `large-v3` result is limited differently: one configuration, one machine type and 10 decodes taken on different occasions and loads, which is why §4 claims a spread and not a rank.

Sample sizes. The diarization corpus is 16 recordings, the streaming comparison is 15 utterances, the cross-instruction-set matrix and its within-host repeat are 60 utterances, and the quality-gate assessment (in-sample) is five meetings. We report intervals and paired tests where the sample supports them and say so explicitly where it does not. The quality gate's recall is 1/1; that is an existence proof of detection, not a rate.

Hosts and timing. Every timing figure is a property of one machine under one load, and Table 1 records both. Figures from different hosts are never merged. Two timing measurements were taken on contended hosts and are marked where they appear: RTF is omitted entirely for §7, and §4 and §9 carry the caveat in place.

Software versions. All faster-whisper results use version 1.2.1 with CTranslate2 4.8.1. The mechanism findings in §5 depend on that library's specific handling of two decode parameters and may not hold for other implementations of the same model. One pair of artifacts was produced under different NumPy versions and their absolute WERs are consequently not comparable; the comparison within each artifact, which is the question asked, is unaffected.

Fitted versus held-out thresholds. The diarization clustering threshold (1.2) was chosen from a sweep on four of the 16 AMI meetings reported here and confirmed on the full set, so the headline DER is in-sample for that parameter; the same threshold scores 42.13% on 15 VoxConverse recordings, where a lower value is better. The attribution guard's duration-scaled formulation (§10) was selected after observing a flat formulation's false alarms on the same corpora, and the quality gate's thresholds (§11) were fixed against the five meetings they are reported on. Neither is a held-out evaluation and we do not present them as one.

Statistical scope. Intervals and p-values are computed within a section and are not adjusted across sections. Repeated decodes on one machine type are not independent samples of a population of machines. The paired analyses use 200 utterances or 16 recordings, which can resolve only large effects. Neither unit is independent: the utterances come from 33 speakers and the recordings from 4 AMI series, 3 of them scenario-based; §6 and §10 report the speaker- and series-level sensitivity, with the series-level interval being very wide.

Single system. Every finding is measured on one system. The mechanism-level results (§5) are properties of a widely used inference library rather than of our code, and the architectural results (§11, §12) are properties of design patterns rather than of an implementation, but we have not verified either claim on a second system.

15 Limitations and Future Work

No user study. This paper reports no human-subjects evaluation, and neither did its predecessor. Everything here is a technical measurement. The claim that a fabricated clause is worse for a user than a dropped word is argued from the nature of the two errors, not measured; the claim that a guard which fires rarely is more useful than one which fires often is a design principle we hold, not a finding. A text-entry study measuring task-level throughput *and* correction cost on composed prose — the quantity that recognition-accuracy numbers structurally omit [18] — remains the most valuable next step, and is more valuable for the accessibility paths than for the mainstream one. Foley et al. [10] compare speech recognition with touchscreen typing on composition and transcription tasks and show the kind of task-level measurement we did not make.

Insertion rate in the wild. Two arms in this study varied between identical decodes — `large-v3` at its defaults and `base.en` with conditioning off — and in both, the error counts that moved between identical decodes were insertions while the aggregate substitution and deletion counts stood still. That is read speech. We have not measured the rate at which fabricated content reaches a real user's editor during real dictation, which is the number that would justify or refute changing a default for everyone.

The cross-decode detector on more corpora. The strongest result in §11 rests on two meetings at the extremes (16× and 1.008×). Establishing the distribution of the ratio on healthy long-form audio — and therefore a principled flag point rather than our chosen 3× — needs a corpus of recorded meetings we do not have and cannot redistribute. It would also need the quality gate's repeat-run signal, which has an as yet unmerged extension to multi-word loops (§11), tested on real multi-word collapses and on held-out healthy meetings, with false alarms counted. We measured no confidence-based detector for comparison.

A held-out engine comparison. The most valuable single follow-up is to re-run §4's matrix on a corpus that is outside all three engines' disclosed training data, with the precision of every engine recorded, which would convert a benchmark-selection warning into a capability ranking on these CPU configurations. Eight configurations is also not the field, and we report no repeated-decode result for the non-Whisper engines — extending §5's test to transducer architectures is the natural companion experiment, since nothing here establishes whether they share the failure mode.

Per-utterance alignments and controlled environments. The archive stores aggregate substitution, deletion and insertion counts, not per-utterance hypotheses, so the insertion-only reading of §5 is established at the aggregate level only; archiving the hypotheses of future repeats would settle it. Likewise §7 needs a design that varies one environmental factor at a time, with repeats on each runner.

Non-English. Every measurement is English, on English-only checkpoints where the checkpoint offers one. Nothing here transfers to multilingual decoding without being redone.

16 Conclusion

Running speech recognition on the user's own machine removes the network, the provider and the retained audio — and with them the mechanisms that would otherwise notice when the system is wrong. We measured a shipping on-device dictation and meeting-transcription system and found its most damaging failures to be silent ones: a decoder configuration that returns a different transcript each time it is asked and whose run-to-run variation in aggregate error counts is entirely in insertions — so that a single decode cannot place its largest checkpoint in an accuracy ranking; a configuration trap in which a user-supplied hint voids a tuned parameter without saying so; and a long-form decode collapse that reported success and led the system to delete its own recording. We also found that two of our own apparent effects were not resolved when re-scored per item, and that our most striking cross-engine comparison was confounded by the engines' differing relationship to the benchmark's training split. We report those retractions alongside the results, because the discipline that caught them is a larger part of the contribution than any single number.

The practical summary is short. Where a second decode already exists, compare the two. Count insertions separately from other errors. Measure the configuration you actually run, including the one you fall back to. Publish per-item distributions. And never let an irreversible action be gated on the absence of an exception.

Availability and Reproducibility

YazSes is open-source under Apache-2.0 at `https://github.com/MSKazemi/yazses`, with documentation at `https://mskazemi.com/yazses/`; the system itself is described in [24].

Public. The benchmark harness (`paper/benchmark/`) and the archive of JSON result artifacts (`paper/results/`, including the exploratory probes, their console logs and a generated `MANIFEST.md` naming, for every artifact, the producing script and the machine) are in the public repository. Each artifact carries the CPU model, OS, library versions and load average of the run that produced it, and, where recorded, its invoking command line; the corpus digest is recorded only in some later probe artifacts. Of the 80 archived artifacts that carry a provenance block, 48 record no invoking command line, so for most of the older results the exact rerun command has to be reconstructed from the harness and the `MANIFEST.md`.

Not public. The LaTeX source of this manuscript and its generator scripts (which read the public artifacts and emit the macros, tables and figures) are private working files, so each value can be re-derived from the public artifacts but our generator cannot be re-run by a reader. The long-form incident of §11 is documented only in the project's release notes, source and tests; the recording and its transcripts are private.

Obtained separately, and limits. The corpora (LibriSpeech, AMI, VoxConverse) are not redistributed and must be obtained from their publishers; the short-form artifacts record the deterministic selection rule and the number of utterances rather than the utterance ids (the per-utterance probe lists them; the diarization artifacts name each recording), and the CI-runner artifacts of §7 record no corpus digest. Several runs were taken on rented compute that no longer exists, so they cannot be repeated on the same hardware, and one earlier engine-matrix result was overwritten before archiving (§4). No audio, transcript or personal speech data of any user appears in the archive.

References

[1] Max Bain, Jaesung Huh, Tengda Han, and Andrew Zisserman. WhisperX: Time-accurate speech transcription of long-form audio. In *Interspeech 2023*, pages 4489–4493, 2023. doi:10.21437/Interspeech.2023-78. arXiv:2303.00747.

[2] Mateusz Barański, Jan Jasiński, Julitta Bartolewska, Stanisław Kacprzak, Marcin Witkowski, and Konrad Kowalczyk. Investigation of Whisper ASR hallucinations induced by non-speech audio. In *ICASSP 2025 – IEEE International Conference on Acoustics, Speech and Signal Processing*, 2025. doi:10.1109/ICASSP49660.2025.10890105. arXiv:2501.11378.

[3] Kurian Benoy. whisper-normalizer: A python package for text standardization. `https://github.com/kurianbenoy/whisper_normalizer`, 2024. Implements the Whisper EnglishTextNormalizer. Version not recorded in the artifacts. Accessed September 2026.

[4] Maximilian Bisani and Hermann Ney. Bootstrap estimates for confidence intervals in ASR performance evaluation. In *2004 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 1, pages I–409–I–412. IEEE, 2004. doi:10.1109/ICASSP.2004.1326009.

[5] Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, Vasilis Karaiskos, Wessel Kraaij, Melissa Kronenthal, Guillaume Lathoud, Mike Lincoln, Agnes Lisowska, Iain McCowan, Wilfried Post, Dennis Reidsma, and Pierre Wellner. The AMI meeting corpus: A pre-announcement. In *Machine Learning for Multimodal Interaction (MLMI 2005)*, volume 3869 of *Lecture Notes in Computer Science*, pages 28–39. Springer, 2006. doi:10.1007/11677482_3.

[6] Yafeng Chen, Siqi Zheng, Hui Wang, Luyao Cheng, Qian Chen, and Jiajun Qi. An enhanced Res2Net with local and global feature fusion for speaker verification. In *Proc. Interspeech 2023*, pages 2228–2232, 2023. doi:10.21437/Interspeech.2023-1294. arXiv:2305.12838. The ERes2Net speaker embedding used by the diarizer measured here.

[7] Joon Son Chung, Jaesung Huh, Arsha Nagrani, Triantafyllos Afouras, and Andrew Zisserman. Spot the conversation: Speaker diarisation in the wild. In *Proc. Interspeech 2020*, pages 299–303, 2020. doi:10.21437/Interspeech.2020-2337.

[8] Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961. doi:10.1080/01621459.1961.10482090.

[9] Jonathan G. Fiscus, Jerome Ajot, Martial Michel, and John S. Garofolo. The rich transcription 2006 spring meeting recognition evaluation. In *Machine Learning for Multimodal Interaction (MLMI 2006)*, volume 4299 of *Lecture Notes in Computer Science*, pages 309–322. Springer, 2006. doi:10.1007/11965152_28. Defines the diarization error rate and the `md-eval` scoring convention.

[10] Margaret Foley, Géry Casiez, and Daniel Vogel. Comparing smartphone speech recognition and touchscreen typing for composition and transcription. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pages 1–11, 2020. doi:10.1145/3313831.3376861.

[11] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1904.09751.

[12] Nat Jeffries, Evan King, Manjunath Kudlur, Guy Nicholson, James Wang, and Pete Warden. Moonshine: Speech recognition for live transcription and voice commands, 2024. arXiv:2410.15608. `https://arxiv.org/abs/2410.15608`.

[13] Hui Jiang. Confidence measures for speech recognition: A survey. *Speech Communication*, 45(4):455–470, 2005. doi:10.1016/j.specom.2004.12.004.

[14] k2-fsa. sherpa-onnx: Speech-to-text, text-to-speech and speaker diarization using Next-gen Kaldi. `https://github.com/k2-fsa/sherpa-onnx`, 2025. Installed version and model-file revisions not recorded in the diarization artifacts; accessed September 2026.

[15] Allison Koenecke, Anna Seo Gyeong Choi, Katelyn X. Mei, Hilke Schellmann, and Mona Sloane. Careless Whisper: Speech-to-text hallucination harms. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 1672–1681, 2024. doi:10.1145/3630106.3658996; arXiv:2402.08021.

[16] Danni Liu, Gerasimos Spanakis, and Jan Niehues. Low-latency sequence-to-sequence speech recognition and translation by partial hypothesis selection. In *Interspeech 2020*, pages 3620–3624, 2020. doi:10.21437/Interspeech.2020-2897. arXiv:2005.11185.

[17] Dominik Macháček, Raj Dabre, and Ondřej Bojar. Turning Whisper into real-time transcription system. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 17–24, 2023. doi:10.18653/v1/2023.ijcnlp-demo.3. arXiv:2307.14743. Applies the LocalAgreement streaming policy to Whisper.

[18] I. Scott MacKenzie and R. William Soukoreff. Text entry for mobile computing: Models and methods, theory and practice. *Human-Computer Interaction*, 17(2–3):147–198, 2002. doi:10.1207/S15327051HCI172&3_2.

[19] NVIDIA. parakeet-tdt-0.6b-v2. Model card, `https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2`, 2025. Lists the training data of the exact checkpoint measured here, including LibriSpeech (960 hours). The revision loaded in the measurements is not recorded; the repository head on 2026-10-28 is ae9ad07, last modified 2026-06-29, before the measurements. Accessed October 2026.

[20] OpenNMT. CTranslate2: Fast inference engine for transformer models. `https://github.com/OpenNMT/CTranslate2`, 2024. Version 4.8.1 throughout this paper. Accessed September 2026.

[21] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An ASR corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210. IEEE, 2015. doi:10.1109/ICASSP.2015.7178964.

[22] Alexis Plaquet and Hervé Bredin. Powerset multi-class cross entropy loss for neural speaker diarization. In *Proc. Interspeech 2023*, pages 3222–3226, 2023. doi:10.21437/Interspeech.2023-205. The segmentation-3.0 model used by the diarizer measured here; arXiv:2310.13025.

[23] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 28492–28518, 2023. arXiv:2212.04356.

[24] Mohsen Seyedkazemi Ardebili. YazSes: An offline, privacy-first, cross-platform hold-to-talk voice-dictation system, 2026. doi:10.48550/arXiv.2607.28878. arXiv:2607.28878. The companion system paper.

[25] Mohsen Seyedkazemi Ardebili. YazSes: offline hold-to-talk voice dictation. `https://github.com/MSKazemi/yazses`, 2026. Apache-2.0. The benchmark harness and every artifact cited here live under `paper/`. Archive concept DOI 10.5281/zenodo.21856271.

[26] Ilya Stupakov. onnx-asr: Automatic speech recognition in Python using ONNX models. `https://github.com/istupakov/onnx-asr`, 2026. Version 0.12.0, the version recorded in the artifacts; the Parakeet TDT runtime measured here.

[27] SYSTRAN. faster-whisper: Faster Whisper transcription with CTranslate2. `https://github.com/SYSTRAN/faster-whisper`, 2024. Version 1.2.1 throughout this paper. Accessed September 2026.

[28] The AMI Consortium. AMI corpus: meeting IDs explained. `https://groups.inf.ed.ac.uk/ami/corpus/meetingids.shtml`, 2026. Defines the series (a–d) and the scenario-based (S) versus naturally occurring (N) meeting types. Accessed October 2026.

[29] Nik Vaessen. JiWER: Similarity measures for automatic speech recognition evaluation. `https://github.com/jitsi/jiwer`, 2024. Version not recorded in the artifacts. Accessed September 2026.

[30] Jacob O. Wobbrock and Julie A. Kientz. Research contributions in human-computer interaction. *Interactions*, 23(3):38–44, 2016. doi:10.1145/2907069.

[31] Hainan Xu, Fei Jia, Somshubra Majumdar, He Huang, Shinji Watanabe, and Boris Ginsburg. Efficient sequence transduction by jointly predicting tokens and durations. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 38462–38484, 2023. arXiv:2304.06795. The Token-and-Duration Transducer behind Parakeet TDT.